PROJECT DOCUMENTATION

Hirer

AI-powered candidate ranking system

A complete guide written for two audiences: Part A explains Hirer in plain language for anyone, and Part B documents the architecture, scoring model, API, frontend and deployment in full technical depth.

Built for the Redrob Data & AI Challenge 2026

Team member	Role	Programme
Ravada Siddharth	ML Engineer / Lead	B.Tech CSE (Data Science), MUIT Noida, 2023–2027
Ishan Gupta	Data Engineer	BCA CSE, ITS Ghaziabad, 2023–2026
Ayush Mishra	Frontend / Docs	B.Tech CSE (Data Science), MUIT Noida, 2023–2027
Vinayak Kapoor	Testing & QA	B.Tech CSE (Data Science), MUIT Noida, 2023–2027

github.com/Sidvortex/Hirer
Version 1.0 · October 2026

Contents

How to read this document

Part A — Hirer for everyone

1 What Hirer is
2 The problem it solves
3 What you can do with Hirer
4 How Hirer decides who ranks higher
5 Using the app, step by step
6 Reading the results
7 What Hirer cannot do yet
8 Your data and privacy

Part B — Technical documentation

9 System architecture
10 Data model
11 The scoring model
12 Submission pipeline and validation
13 Resume extraction
14 Backend API
15 Frontend
16 Deployment and operations
17 Testing and quality assurance
18 Known limitations and technical debt
19 Roadmap
20 Team

Appendices

A Keyword lists
B API quick reference
C Glossary

How to read this document

If you are not technical, read Part A. It explains what Hirer does, how it decides, how to use it and how to interpret its results, without any code.

If you are an engineer, evaluator or future contributor, Part B documents every component: the data format, each scoring formula with its exact constants, the resume extractor, the API, the frontend, deployment, testing and known issues. Section 11.13 walks one real candidate through every calculation.

If you only have five minutes, read Sections 1, 4 and 6, then look at Figures 2 and 3.

PART A

Hirer for everyone

No technical background needed. This part explains what Hirer is, who it helps, how it decides who ranks higher, how to use it, how to read its results, and what it cannot do yet.

1. What Hirer is

Hirer is a web application that reads candidate profiles or resumes and ranks people by how well they fit a job. It was built by a four-person student team for the **Redrob Data & AI Challenge 2026**. The challenge task was specific: given 100,000 anonymized candidate profiles and one job description, a Senior AI Engineer role at Redrob AI, pick the 100 best candidates and explain each choice.

The challenge entry then grew into a fuller tool. Besides the challenge's own data format, Hirer can read real resumes (PDFs and images), rank a whole folder of them at once, or tell an individual job seeker which kinds of roles suit them best.

> **In one sentence:** Hirer looks at every candidate the way three different reviewers would. One checks skills and experience, one checks whether the profile means the same thing as the job, and one checks whether the person is active and available. Hirer then blends their opinions into one ranked list, with a short reason next to every name.

2. The problem it solves

For recruiters

A single job opening can attract hundreds or thousands of applications, and reading each one carefully takes minutes. The usual shortcut is a keyword filter, which is fast but brittle: a candidate who writes "dense retrieval" gets rejected by a filter looking for "vector search", even though both describe very similar work. And even a perfect profile is of little use if the person has not logged in for months or never replies to recruiters.

For job seekers

Job titles vary wildly between companies, and it is often unclear which roles your experience actually fits. Applying blindly wastes time on both sides.

Hirer's approach

Instead of relying on one kind of evidence, Hirer combines three: what the person can do, what their profile actually means, and how likely they are to respond. It also always shows why a candidate ranked where they did, so a human can check the reasoning instead of trusting a black box.

3. What you can do with Hirer

Hirer has three modes. You choose one on the "Choose your mode" screen.

Mode	Who it is for	What you upload	What you get back
Bulk JSON ranking	Recruiters and challenge evaluators	A `candidates.jsonl` file of structured profiles (the challenge format)	A ranked list, the top three highlighted, a reason for every candidate, and a CSV download

Mode	Who it is for	What you upload	What you get back
Single resume	Job seekers	One resume as a PDF, JPG or PNG	The skills and experience Hirer found, your best-matching role, and a match percentage for 20 roles across 12 fields
Bulk resume ZIP	Recruiters with a folder of resumes	A ZIP file of PDF or image resumes	Every resume read automatically, then ranked, with names and a CSV download

> **Live version limits:** The free online version accepts up to 1,000 profiles per JSON file and up to 50 resumes (50 MB) per ZIP. When Hirer is run on your own computer there are no limits.
>
> The free server sleeps when nobody is using it, so the very first visit can take up to a minute. Hirer starts waking the server the moment the opening animation begins, so by the time you reach the upload screen it is usually ready.

4. How Hirer decides who ranks higher

Think of Hirer as a panel of three reviewers. Each one looks at the same candidate from a different angle and gives a score. Their scores do not count equally.

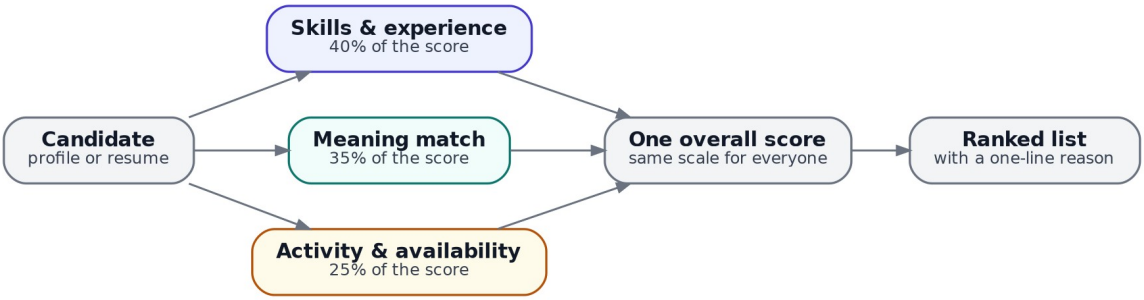


Figure 1. Three reviewers, three weights, one ranked list.

Reviewer 1: skills and experience (40%)

This reviewer checks the profile against the job's requirements. For the challenge job that means skills like embeddings, vector databases, semantic search, Python, NLP and ranking evaluation. It also checks whether the person's years of experience fall in the 5 to 9 year range the job asks for, whether their current title is relevant, and the tier of their college.

Reviewer 2: meaning match (35%)

This reviewer uses an AI language model. The model turns both the job description and the candidate's profile into long lists of numbers that capture meaning, so two texts that mean similar things end up with similar numbers even when they use different words. That is how Hirer recognises that "dense retrieval" and "vector search" are close cousins.

Reviewer 3: activity and availability (25%)

A great profile matters less if the person is not looking. This reviewer asks: were they active recently? Are they marked as open to work? Do they reply to recruiters, and how quickly? Do they have recent GitHub activity? How long is their notice period?

Blending the three

Each reviewer's scores are first put on the same 0 to 1 scale across everyone in the upload, so no reviewer can dominate just because its numbers happen to be bigger. Then the three are blended using the 40 / 35 / 25 weights, everyone is sorted from highest to lowest, and Hirer writes a one-line reason for each person.

> **A real example:** In the 1,000-person sample dataset, the top candidate is an AI Research Engineer with 6.3 years of experience. They sit right in the 5 to 9 year sweet spot, their title matches what the job wants, their profile mentions 9 of the job's required skills, they are open to work, and they reply to 79% of recruiter messages. They were held back a little because their profile mentions OpenCV (a computer vision tool, which the job specifically does not want) and because they have not been active for several months.

5. Using the app, step by step

1. **Open the site.** A handwritten "Hirer" animation plays while the server wakes up in the background. Press any key to continue.
2. **Home screen.** Click **Start Ranking**. The "about the team" link in the corner shows who built Hirer.
3. **Choose your mode.** Bulk JSON, Single resume, or Bulk resume ZIP.
4. **Upload your file.** Drag and drop it or click to browse. In the JSON and ZIP modes you can also adjust how much each reviewer counts (the three sliders must add up to 100) and how many top candidates to keep.
5. **Wait for processing.** A progress ring shows the percentage, the real processing steps appear underneath, and a rotating set of jokes keeps you company.
6. **Read the results.** The top three candidates appear as stacked cards, followed by the full ranked table with a reason for each person.
7. **Download.** Click **download csv** to save the ranked list as a spreadsheet-friendly file.

In Single resume mode the flow is shorter: upload one resume and you go straight to a results page showing what Hirer understood about you, your best match, and a percentage for every role. Use the filter buttons to show only one field, such as Data or DevOps.

6. Reading the results

The score is relative, not a grade

A score compares a candidate with the other candidates in the same upload. A score of 58% means "near the top of this group", not "58% qualified". If you upload a different group, the same person can receive a different score.

Why the online version’s scores look low

On the free online server the meaning-match reviewer is switched off to save memory, so nobody can earn its 35% share. The highest possible score there is 65%, which is why the best candidate typically shows something like 58%. In Bulk resume ZIP mode the ceiling is lower still, 40%, because resumes contain no activity information, so only the skills reviewer can tell resumes apart.

The reason line

Each candidate gets a short explanation such as: “AI Research Engineer with 6.3 yrs exp; 9 JD skill matches; open to work; high recruiter response rate (0.79).” It lists the current title and experience, how many of the job’s key skills appear in the profile, and any strong signals: open to work, a GitHub activity score above 50, or a recruiter response rate above 0.7.

Single resume matches

Each of the 20 roles gets a percentage and a label: **Strong match** above 60%, **Good match** from 40% to 60%, and **Partial match** below 40%. Under each role Hirer shows how many of that role’s key skills it found in your resume.

7. What Hirer cannot do yet

- **It reads resumes using rules, not full understanding.** Unusual layouts such as two columns, tables or heavy graphics can confuse it.
- **Scanned or photographed resumes need extra software.** Reading text from an image (OCR) needs a program the current free server cannot install. Until the planned move to a new server, use a text-based PDF. A quick fix: open your resume in a browser, press Print, and choose “Save as PDF”.
- **It only knows the jobs it was set up with:** the challenge job for ranking, and 20 role profiles for job seekers.
- **The “rerank for a different job” wheel on the results page is an approximate preview.** It does not re-run the full analysis against a new job description.
- **It does not judge soft skills, potential, or anything not written down.**
- **Activity scores depend on the date you run it.** The challenge data is from 2025 to 2026, so running it months later makes everyone look less active.
- **Fairness.** Some signals, like the tier of a person’s college, can reflect privilege rather than ability. Hirer gives education only a small weight (about 4% of the final score), but it is still there. Hirer should support human judgment, not replace it.

8. Your data and privacy

- **No accounts and no database.** Uploaded files are processed on Hirer’s own server and kept in temporary storage, which is wiped whenever the server restarts or is redeployed. Files are not individually deleted straight after processing.
- **Nothing is sent to outside AI services.** All reading and scoring happens on Hirer’s server, including the AI language model.

- **Your files are not used to train anything.**
- **Results are identified only by a long random ID** that is never shown in the interface.
- **Only upload resumes you have permission to process,** especially when using the bulk modes.

PART B

Technical documentation

For engineers, evaluators and future contributors. Every number in this part comes from the source code as of October 2026, and the worked examples were produced by running that code on the 1,000-candidate sample in data/candidates_small.jsonl.

9. System architecture

Hirer has three entry points that converge on one scoring core. Structured profiles go straight to scoring; resumes first pass through an extractor that converts free text into the same candidate schema. A command-line pipeline uses the same scoring code without any server, and that is what produces the challenge submission.

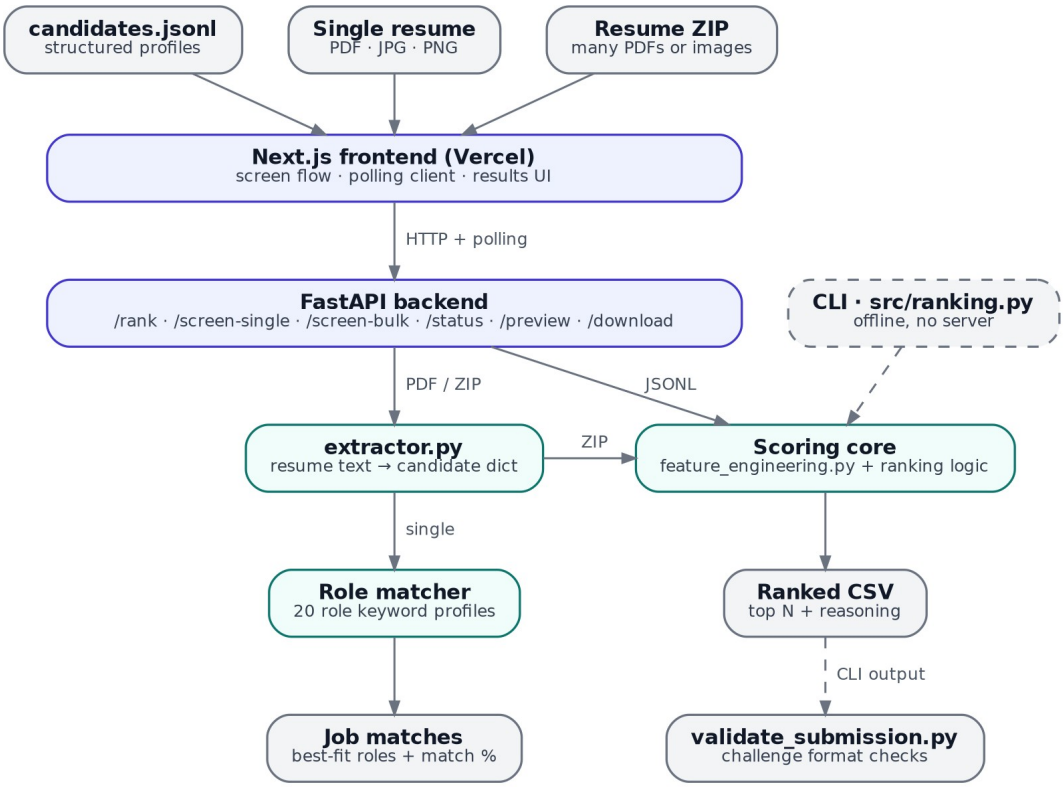


Figure 2. System architecture. Solid arrows are the web app; dashed arrows are the offline CLI path.

Component	Location	Technology	Responsibility
Frontend	`hirer-frontend/`	Next.js 16.2, React 19.2, TypeScript, Tailwind CSS, Framer Motion, lucide-react	Screen flow, uploads, polling, results UI
Backend API	`hirer-backend/main.py`	FastAPI, Uvicorn	Endpoints, in-memory job store, background jobs, role matching
Scoring core	`feature_engineering.py`, `ranking.py`, `utils.py`	pandas, NumPy, scikit-learn, sentence-transformers	Feature scores, semantic similarity, normalization, ranking, reasoning
Resume extractor	`hirer-backend/extractor.py`	pdfplumber, pytesseract, pdf2image, Pillow	Resume file → candidate dict
CLI pipeline	`src/ranking.py`	Python	Offline generation of `submission.csv`

Component	Location	Technology	Responsibility
Validator	`validate_submission.py`	Python standard library	Challenge format checks
Exploration	`notebooks/exploration.ipynb`	Jupyter, matplotlib	Exploratory data analysis

9.1 Repository layout

```
Hirer/
├── data/               schema, samples, candidates_small.jsonl (1,000)
├── notebooks/          exploration.ipynb
├── src/                CLI: ranking.py, feature_engineering.py, utils.py
├── hirer-backend/      main.py, extractor.py, scoring copies, Dockerfile
├── hirer-frontend/     Next.js app: app/page.tsx, components/ui/
├── output/             generated submission.csv
├── start.sh            one-command local start (backend + frontend)
├── validate_submission.py
├── submission_metadata.yaml
└── README.md
```

9.2 Runtime versions

Use **Python 3.11** for the backend and CLI. Python 3.14 (the default on some hosts and on rolling-release Linux distributions) fails to build older pandas releases and lacks wheels for the PyTorch stack behind sentence-transformers. The frontend needs Node.js 18 or newer.

10. Data model

10.1 Input format

Candidates arrive as **JSON Lines**: one JSON object per line. `load_candidates()` also reads gzip-compressed files (`.jsonl.gz`). The upload form accepts `.json` too, but the file must still contain one object per line; a pretty-printed JSON array such as `sample_candidates.json` will not parse.

10.2 Candidate schema

Block	Key fields	Used by Hirer for
`candidate_id`	`CAND_` followed by 7 digits	Output identity; tie-breaking
`profile`	headline, summary, current_title, years_of_experience, location, current_company, current_industry	Keyword text, semantic text, experience score, title score
`career_history[]`	company, title, start_date, end_date, duration_months, description, industry	Keyword text (all jobs); semantic text (first two jobs)
`education[]`	institution, degree, field_of_study, start_year, end_year, grade, tier	Education score (tier only)

Block	Key fields	Used by Hirer for
`skills[]`	name, proficiency, endorsements, duration_months	Skill names in keyword and semantic text
`certifications[]`	name, issuer, year	Names in keyword text
`languages[]`	language, proficiency	Not used
`redrob_signals`	23 platform activity fields	7 fields feed the behavioral score

10.3 Platform signals: used and unused

Hirer uses 7 of the 23 `redrob_signals` fields: `last_active_date`, `open_to_work_flag`, `recruiter_response_rate`, `avg_response_time_hours`, `github_activity_score`, `profile_completeness_score` and `notice_period_days`. Section 11.8 describes exactly how each is scored.

The other 16 are unused: `signup_date`, `profile_views_received_30d`, `applications_submitted_30d`, `skill_assessment_scores`, `connection_count`, `endorsements_received`, `expected_salary_range_inr_lpa`, `preferred_work_mode`, `willing_to_relocate`, `search_appearance_30d`, `saved_by_recruiters_30d`, `interview_completion_rate`, `offer_acceptance_rate`, `verified_email`, `verified_phone` and `linkedin_connected`. Several are strong candidates for future work, especially `willing_to_relocate` (the job is in Pune or Noida), `interview_completion_rate` and `skill_assessment_scores`.

10.4 Output format

`submission.csv` has exactly four columns in this order: `candidate_id,rank,score,reasoning`, with exactly 100 data rows. Section 12.2 lists every rule the official validator enforces.

11. The scoring model

11.1 Overview

Every candidate receives three component scores: a **skill composite S**, a **semantic score E** and a **behavioral score B**. Each is min-max normalized across all candidates in the run, then combined:

```
final = 0.40 · N(S) + 0.35 · N(E) + 0.25 · N(B)

N(x)  = (x − min x) / (max x − min x)
        computed over every candidate in the run
        (if max x = min x, every N(x) is 0)
```

The weights are configurable in the web app (the three sliders) and default to 0.40 / 0.35 / 0.25. The CLI uses the defaults.

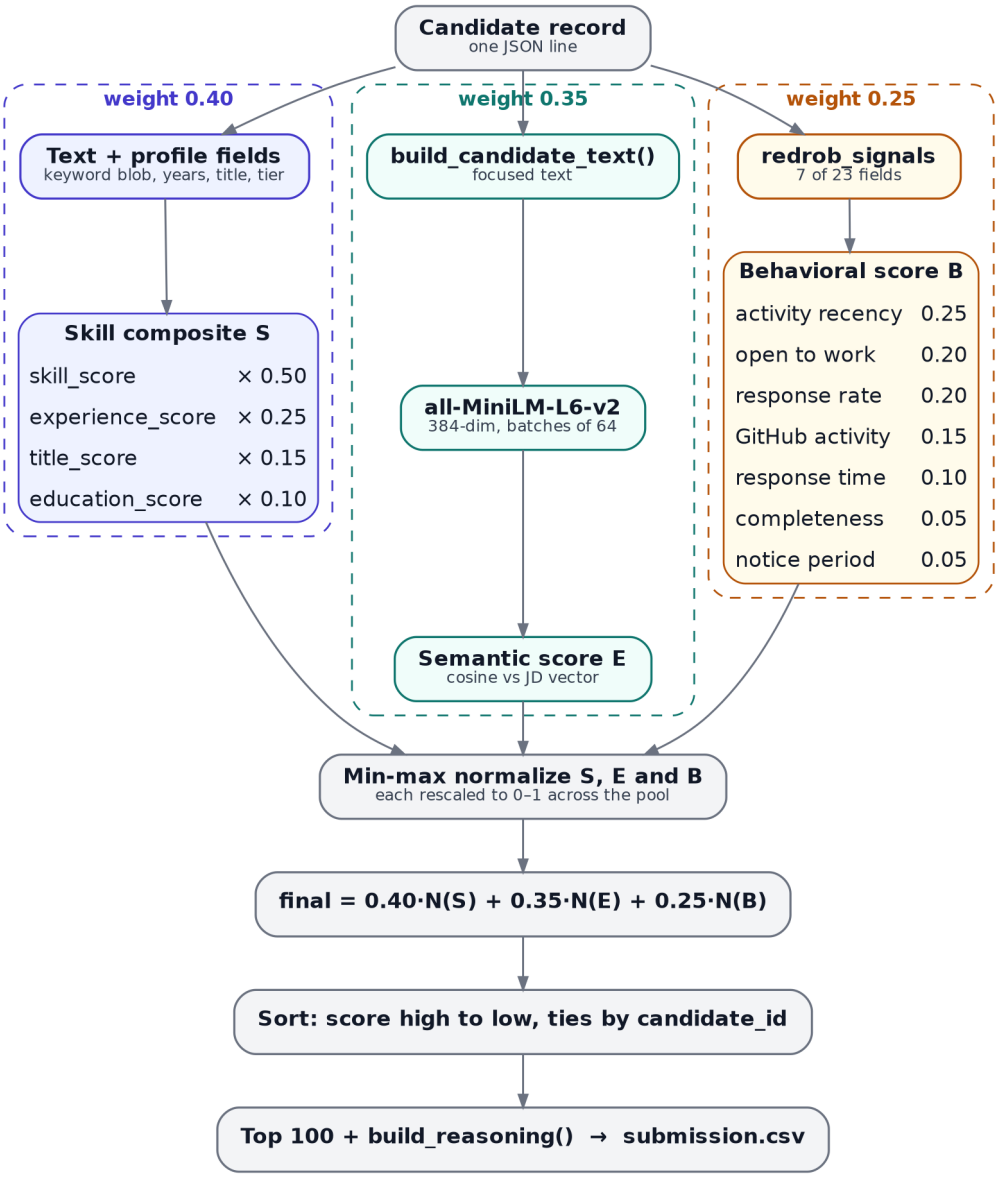


Figure 3. How one candidate record becomes a ranked row in submission.csv.

11.2 Two views of the same candidate

Hirer builds two different text representations, because keyword matching and embedding models want different inputs.

Function	Contents	Used for
`_get_full_text()`	Headline, summary, current title, every job title and description, every skill name, every certification name	Keyword matching (skill_score and the reasoning line)
`build_candidate_text()`	Headline, summary, current title, a “Skills: …” list, and only the first two jobs with the first 300 characters of each description	Semantic embedding. Kept short because the embedding model truncates long inputs at 256 word pieces

11.3 Keyword skill score

`skill_score()` searches the lowercase keyword text for three lists taken from the job description: **27 must-have** terms, **16 nice-to-have** terms and **7 negative signals** (full lists in Appendix A).

```
raw = (2 × must_hits + 0.5 × nice_hits) / 54     # 54 = 27 × 2
raw = min(raw, 1.0)
if negative_hits > 0:
    raw = raw × max(0.6, 1 − 0.1 × negative_hits)  # floor 0.6×
```

Matching is plain substring search, which is fast but has side effects worth knowing:

- `cv` matches inside “OpenCV”, and would also match a summary that says “see my CV”. In the 1,000-candidate sample, 47 candidates (4.7%) trip this negative signal, including the top-ranked candidate.
- `bert` would match “Robert”, and `ocr` matches inside any word containing those letters.
- Overlapping terms double count: `transformer` also matches inside `sentence-transformers`, and `retrieval` also matches inside `information retrieval`.

11.4 Experience score

The job asks for 5 to 9 years. `experience_score()` maps `profile.years_of_experience` onto bands; over-experienced candidates score below the target band but above clearly junior ones.

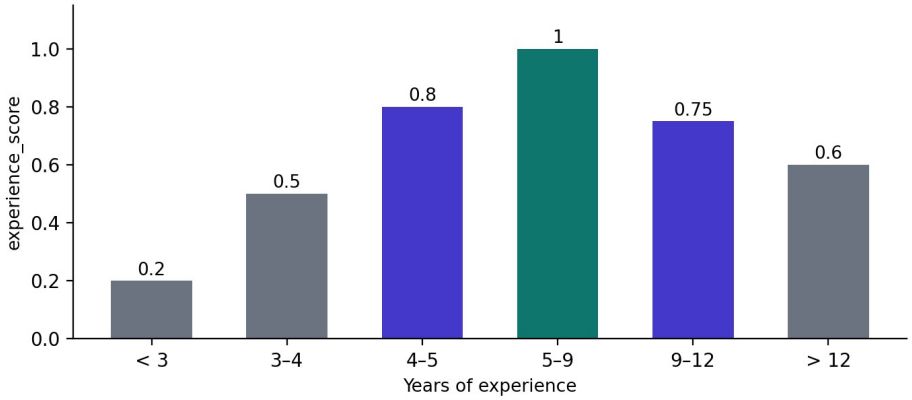


Figure 4. experience_score by years of experience.

11.5 Title score

`title_score()` returns **1.0** if the current title contains any of 11 preferred titles (for example “ml engineer”, “search engineer”, “research engineer”, “applied scientist”), **0.6** if it contains “engineer” or “scientist”, **0.3** for “developer” or “analyst”, and **0.1** otherwise.

11.6 Education score

Best tier across all degrees	tier_1	tier_2	tier_3	tier_4	unknown or missing
education_score	1.0	0.75	0.5	0.3	0.4

11.7 Skill composite

```
S = 0.50 × skill_score      + 0.25 × experience_score
  + 0.15 × title_score      + 0.10 × education_score
```

11.8 Behavioral score

`behavioral_score()` is a weighted average of seven platform signals. The weights sum to 1.0.

Signal	Field	Scoring	Weight
Activity recency	`last_active_date`	≤ 7 days → 1.0; ≤ 30 → 0.8; ≤ 90 → 0.5; older → 0.1 (unparseable date counts as 365 days)	0.25
Open to work	`open_to_work_flag`	true → 1.0; false → 0.3	0.20
Recruiter response rate	`recruiter_response_rate`	Raw value 0–1 (default 0.5)	0.20
GitHub activity	`github_activity_score`	score ÷ 100; −1 (no GitHub linked) → 0.3	0.15
Response time	`avg_response_time_hours`	≤ 2 h → 1.0; ≤ 12 → 0.8; ≤ 24 → 0.6; ≤ 72 → 0.3; slower → 0.1	0.10
Profile completeness	`profile_completeness_score`	value ÷ 100	0.05
Notice period	`notice_period_days`	≤ 30 → 1.0; ≤ 60 → 0.7; ≤ 90 → 0.4; longer → 0.2	0.05

> **Date drift:** Recency is measured against `date.today()`, so the same file ranks differently depending on when it is run. The sample's `last_active_date` values run from 29 Sep 2025 to 27 May 2026. Run on 1 Jul 2026, 27.2% of candidates count as active in the last 90 days; run on 3 Oct 2026, none do, and the most heavily weighted behavioral signal becomes the same 0.1 for everyone. A fixed reference date (for example the latest `last_active_date` in the file) would make results reproducible.

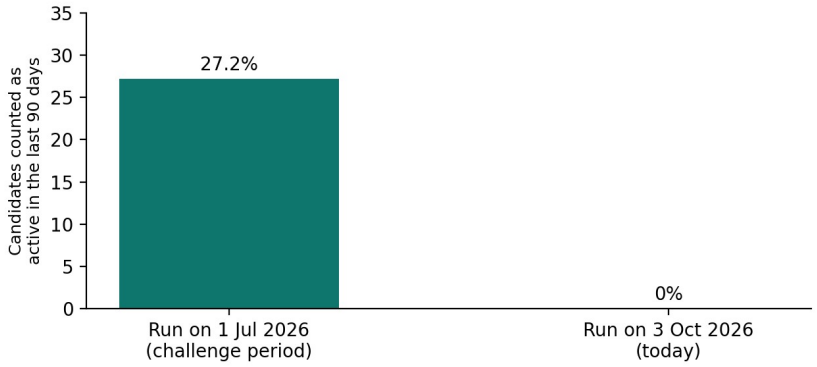


Figure 5. Share of the 1,000-candidate sample counted as recently active, by run date.

11.9 Semantic score

The semantic component uses **all-MiniLM-L6-v2** from sentence-transformers: a 6-layer MiniLM model that maps text to a **384-dimensional** embedding, trained so that texts with similar

meaning land close together. Hirer embeds the job description once, embeds every candidate's `build_candidate_text()` in batches of 64 to keep memory flat, and scores each candidate by **cosine similarity** to the job vector (scikit-learn).

If sentence-transformers is not installed, or the CLI is run with `--no-semantic`, every candidate gets 0.5. The web app's live server runs with semantic scoring off to fit in memory.

> **Negation caveat:** The CLI's job text includes lines such as "NOT looking for: pure computer vision, speech, robotics". Embedding models do not reliably understand negation, so those words can pull computer vision profiles closer to the job rather than further away. The keyword penalty in skill_score is what actually pushes them down.

11.10 Normalization and combination

Min-max normalization makes the three components comparable, but it has three consequences that are easy to miss:

- **Scores are relative.** They depend on who else is in the file. Adding or removing candidates shifts everyone.
- **A constant component contributes nothing.** With semantic scoring off, every E is 0.5, `normalize_0_1` returns all zeros, and the maximum possible final score is 0.65. In Bulk ZIP mode, behavioral signals are also identical for everyone (Section 13.3), so the maximum is 0.40. With only two resumes, min-max turns their scores into exactly 1 and 0.
- **Outliers compress everyone else.** One extreme value stretches the range and squeezes the rest of the pool towards zero.

11.11 How much each feature really counts

Multiplying the nested weights gives each feature's nominal share of the final score. Semantic similarity alone (35%) outweighs everything except the full skill composite, which is a strong argument for running with semantic scoring on whenever memory allows. These are nominal shares; normalization rescales each component before the weights apply.

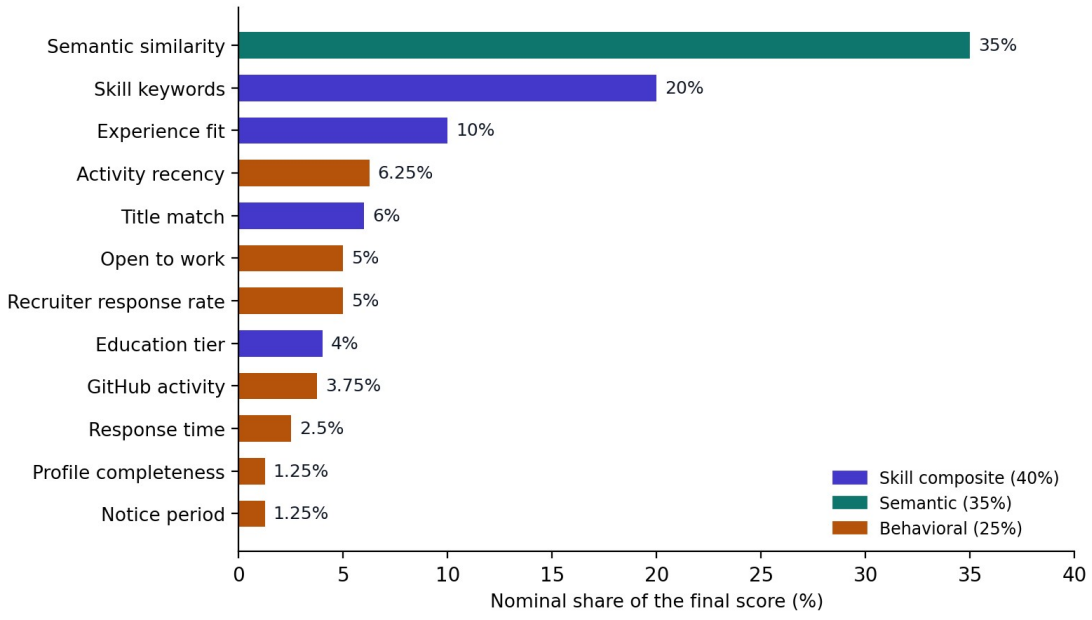


Figure 6. Nominal share of the final score by feature.

11.12 Ranking, tie-breaking and reasoning

Candidates are sorted by score, highest first, and the top N (100 for the challenge) are written out. The official validator requires that **equal scores appear in ascending candidate_id order**. Because scores are rounded to 4 decimal places, ties do happen: on the 1,000-candidate sample the top 100 contained 4 tied pairs. The original `np.argsort` did not order ties, and the validator rejected the output. The fixed version (October 2026) sorts on the rounded score and then the ID:

```
rounded    = np.round(final_scores, 4)
ids        = np.array([c['candidate_id'] for c in candidates])
sorted_idx = np.lexsort((ids, -rounded))   # score desc, then id asc
```

`build_reasoning()` writes the explanation column from a fixed template: current title and years of experience, the number of must-have keywords found, then “open to work” if flagged, “GitHub score N” if above 50, and “high recruiter response rate (x)” if above 0.7.

11.13 Worked example: the top candidate in the sample

Computed on 3 Oct 2026 by running the repository code on `data/candidates_small.jsonl` (1,000 candidates) with semantic scoring off.

Step	Value	How it was computed
Candidate	CAND_0000422	AI Research Engineer, 6.3 years, M.Tech from SRM Chennai (tier_3)
Must-have hits	9	milvus, opensearch, python, ranking, nlp, fine-tuning, transformer, bert, learning to rank
Nice-to-have hits	4	xgboost, pytorch, hugging face, mlops
Negative hits	1	`cv`, matched inside “OpenCV” in the skills list
skill_score	0.3333	(2×9 + 0.5×4) / 54 = 0.3704, × 0.9 penalty
experience_score	1.0	6.3 years is inside 5–9
title_score	1.0	Title contains “research engineer”
education_score	0.5	tier_3
S (skill composite)	0.6167	0.5×0.3333 + 0.25×1.0 + 0.15×1.0 + 0.10×0.5
B (behavioral)	0.4997	Last active 24 Mar 2026, 193 days ago → 0.1; open to work → 1.0; response rate 0.79; 83 h response time → 0.1; no GitHub → 0.3; completeness 83.3 → 0.833; 90-day notice → 0.4
N(S)	0.98	Pool range of S: 0.0950 to 0.6273
N(B)	0.7389	Pool range of B: 0.1748 to 0.6145
N(E)	0	Semantic off: every E is 0.5, so all normalize to 0
Final	0.5767	0.40×0.98 + 0.35×0 + 0.25×0.7389, rank 1 of 1,000

Step	Value	How it was computed
Reasoning		“AI Research Engineer with 6.3 yrs exp; 9 JD skill matches; open to work; high recruiter response rate (0.79).”

12. Submission pipeline and validation

12.1 Running the CLI

```
python src/ranking.py --candidates data/candidates.jsonl --out output/submission.csv

  --candidates   path to .jsonl or .jsonl.gz (required)
  --out          output CSV path (default output/submission.csv)
  --top-n        rows to keep (default 100)
  --no-semantic  skip the embedding model (faster; semantic weight inert)

python validate_submission.py output/submission.csv
```

The pipeline loads all candidates, computes the feature scores, optionally embeds and scores semantic similarity, normalizes and combines, sorts, and writes the CSV with reasoning. The team’s submission metadata reports roughly 3 minutes for 100,000 candidates on a CPU-only laptop with semantic scoring on.

12.2 What the validator enforces

- The file uses a `.csv` extension and is UTF-8 encoded.
- Row 1 is exactly `candidate_id,rank,score,reasoning`.
- Exactly 100 data rows, each with exactly 4 columns.
- Every `candidate_id` matches `CAND_` plus 7 digits, with no duplicates.
- Every rank from 1 to 100 appears exactly once, as a plain integer.
- Every score parses as a float, and scores never increase as rank increases.
- Equal scores are ordered by `candidate_id` ascending.

> **Verified:** Before the tie-break fix, the sample run failed validation with three tie-order errors (ranks 37/38, 58/59 and 82/83). After the fix the validator reports “Submission is valid.” Note that the Bulk ZIP CSV is deliberately not a challenge submission: it uses generated 8-character IDs and extra columns.

13. Resume extraction

`extractor.py` turns a resume file into a candidate dict with the same schema as the challenge data, so the scoring core never needs to know where a candidate came from. It uses no language model: text is pulled out of the file, then a set of regex and keyword rules fills each field.

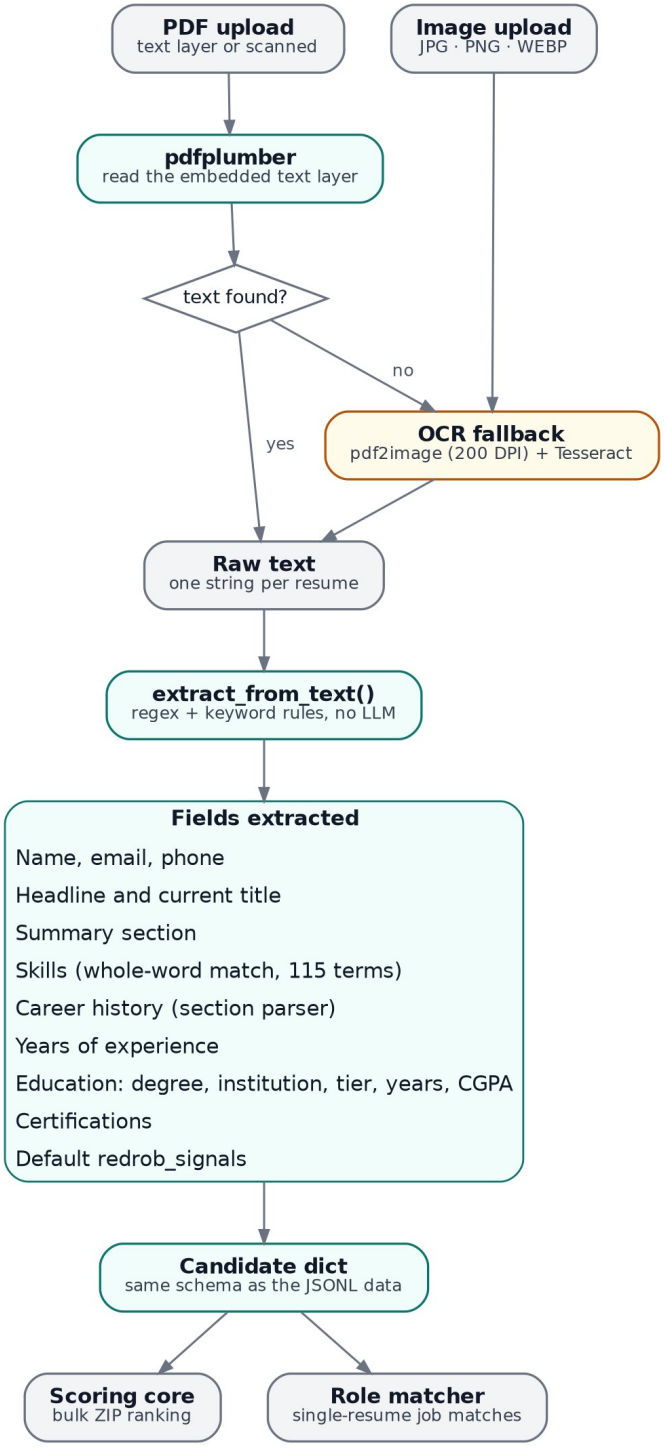


Figure 7. Resume extraction pipeline.

13.1 Getting text out of the file

- **Text PDFs:** `pdfplumber` reads the embedded text layer page by page.
- **Scanned PDFs:** if pdfplumber returns nothing, `pdf2image` renders each page at 200 DPI and `pytesseract` runs Tesseract OCR on it.
- **Images (JPG, PNG, WEBP):** sent straight to Tesseract.

- OCR needs two system packages, `tesseract-ocr` and `poppler-utils`. When they are missing, the functions return an empty string instead of crashing, and the API answers with HTTP 422 and a message suggesting a text-based PDF.

13.2 Field extraction

Field	Method
Name	First 4 lines; a line of 2–4 capitalized words, or an all-caps line of up to 4 words; skips lines containing words like “resume”, “cv” or “profile”
Email, phone	Regular expressions
Headline, current title	First line near the top containing a role word (engineer, developer, analyst, manager, designer…), under 80 characters; otherwise a role line near “present” or “current”
Summary	Up to 4 lines after a Summary, Objective, About or Profile header (max 500 characters); otherwise the first long line
Skills	A 115-term vocabulary matched as **whole words** (so “rag” does not match “average”); “expert in X” style phrases mark proficiency
Career history	Finds the Experience header; each short line containing a year (19xx or 20xx) starts a job; the following lines become title, company, then description; stops at the next section header; up to 6 jobs
Years of experience	An explicit “N years of experience” wins; otherwise current year minus the earliest job start year
Education	11 degree patterns (B.Tech, M.Tech, MBA, PhD, B.Sc…); institution by whole-word lookup, longest name first, across 43 known institutions in three tiers; years from the degree line and the line after it; CGPA written either way round, or a percentage
Location, industry	First of 17 known city names; industry inferred from keyword groups
Certifications	A 13-term keyword list (AWS Certified, Coursera, PMP, CFA…)

13.3 Default platform signals

Resumes carry no platform activity, so `_default_signals()` fills `redrob_signals` with fixed values: open to work, 0.75 response rate, 24-hour response time, active today, no GitHub, 70% completeness and a 30-day notice period. Every resume therefore gets the same behavioral score, **B = 0.79**, which normalizes to zero for everyone. In resume modes the behavioral weight effectively does nothing.

13.4 Fixes made in October 2026

Problem	Effect	Fix
Year regex `(19|20)\d{2}` used a capturing group	`re.findall` returned “20” instead of “2019”, so experience from dates was always 0 and all career and education years were wrong	Non-capturing group `(?:19|20)\d{2}`

Problem	Effect	Fix
Experience used every year in the document	Education years inflated experience (a 2014 graduation counted as work)	Earliest job start year only
Education years read ±200 characters around the degree	Picked up job dates from the section above	Degree line and the next line only
Substring matching for skills and colleges	“rag” matched “average”, “excel” matched “excellent”, “mit” matched “submit”, “nit” matched “community”	Whole-word matching; longest institution name first
CGPA regex expected the number first	“CGPA 8.6” was missed and a stray percentage was used	Accept both orders, search only the degree lines

14. Backend API

The backend is a single FastAPI application in `hirer-backend/main.py`, served by Uvicorn. Long-running work never happens inside a request: uploads start a background job and return a job ID immediately, and the frontend polls for progress.

14.1 Endpoints

Method and path	Purpose	Input	Output
`GET /health`	Liveness check and wake-up ping	None	`{"status": "ok"}`
`POST /rank`	Start a JSONL ranking job	Multipart: `file`, `skill_weight`, `semantic_weight`, `behavioral_weight` (must sum to 1.0 ± 0.01), `top_n`, `use_semantic`	`{job_id, status}`
`GET /status/{job_id}`	Poll progress	None	`{job_id, status, step, pct, steps, error, result}`
`GET /preview/{job_id}`	Results as JSON	`limit` (default 20; the UI asks for 100)	`{results: [...]}`
`GET /download/{job_id}`	Download results	None	`submission.csv`
`POST /screen-single`	Match one resume against 20 roles	Multipart: `file` (.pdf, .jpg, .jpeg, .png, .webp)	Candidate summary, `job_matches`, `best_match`, `top_domain`
`POST /screen-bulk`	Start a resume ZIP ranking job	Multipart: `file` (.zip), the three weights, `top_n` (default 50)	`{job_id, status}`

14.2 Job lifecycle

`POST /rank` saves the upload to `<tmp>/hirer/<job_id>/candidates.jsonl`, creates an entry in the module-level `JOBS` dictionary with status `running`, schedules `run_ranking_job` through FastAPI's `BackgroundTasks`, and returns. Background tasks run after the response has been sent, which is why the endpoint answers instantly even for large files. The job updates its `pct` and `step` fields as it moves through each stage, and finally writes `submission.csv` next to the input and sets status to `done` (or `error` with a message).

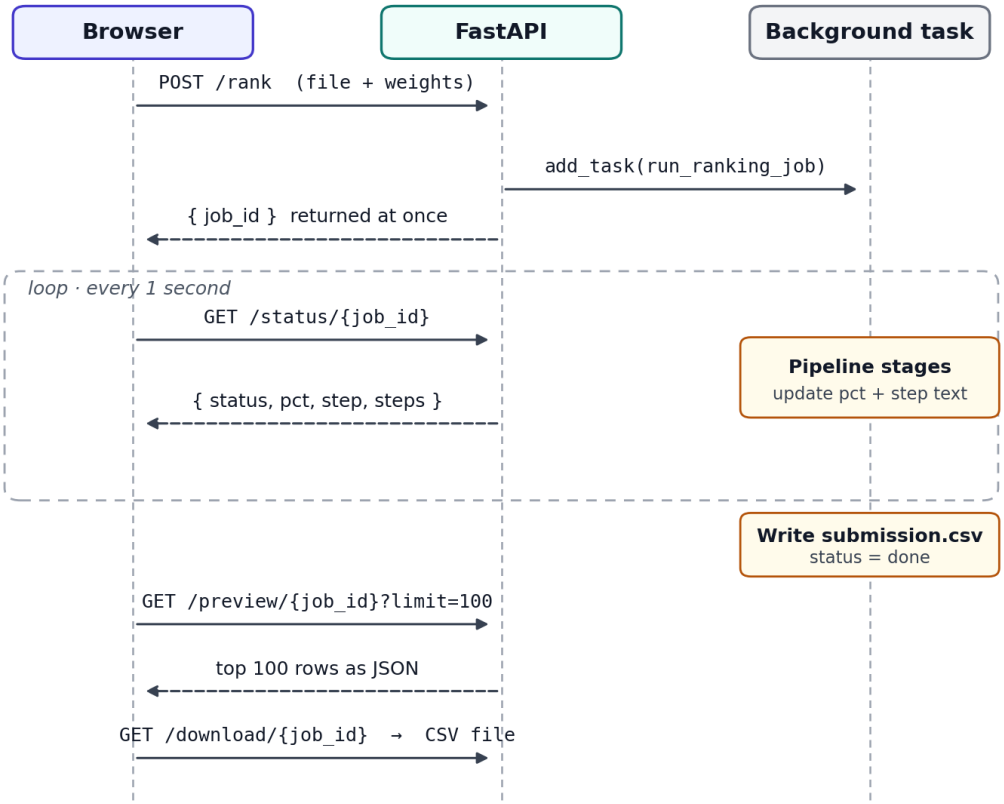


Figure 8. Job lifecycle with polling.

14.3 Concurrency model

`run_ranking_job` is an `async def` that calls `await asyncio.sleep(0)` between stages. Each of those calls hands control back to the event loop so that waiting `/status` requests can be answered. Inside a single long stage, such as scoring a large file or encoding a batch, the event loop is blocked and status requests queue until the stage ends. The resume ZIP job is a plain `def`, which FastAPI runs in a worker thread instead. Making the ranking job a plain `def` too, or moving jobs to a real queue (RQ, Celery, or a separate worker process), would remove the blocking entirely.

14.4 Why polling instead of streaming

The first version streamed progress with Server-Sent Events. On the hosting platform the proxy buffered the stream, so the browser received nothing until the job finished and the loading screen sat at 0% indefinitely. Polling a tiny JSON endpoint once per second works through any proxy, survives brief network drops, and needs no special server configuration. The trade-off is up to a second of extra latency and a little more request overhead, both negligible here.

14.5 Configuration

Variable	Default	Effect
`FRONTEND_URL`	empty	Extra CORS origins, comma separated. Localhost ports 3000 are always allowed.
`JSONL_ENTRY_LIMIT`	0 (no limit)	Maximum non-empty lines accepted by `/rank`; larger files get HTTP 400 with an explanation. Set to 1000 on the live server.
`PDF_COUNT_LIMIT`	0 (no limit)	Maximum files read from a ZIP; extras are skipped with a note in the progress log. Set to 50 live.
`PDF_SIZE_LIMIT_MB`	0 (no limit)	Maximum ZIP upload size; larger uploads get HTTP 400. Set to 50 live.
`PORT`	8000	Port Uvicorn listens on (set by the hosting platform).

> **Fixed in October 2026:** The 1,000-entry JSONL limit was previously shown in the UI but never checked by the server, and every `/status` poll for a resume ZIP job failed because that job's record was missing the `steps` and `result` keys. Both are fixed in the current `main.py`, and the three limits are now environment variables so local runs stay unlimited.

14.6 Single-resume role matching

`/screen-single` does not use the ranking pipeline, because there is nobody to rank against. It extracts the resume, then scores the candidate against 20 role profiles, each a list of 7 to 10 keywords:

```
match = 0.65 × (role keywords found ÷ role keywords)
      + 0.25 × experience_score
      + 0.10                                capped at 0.99

Strong match  > 0.60      Good match  0.40 – 0.60      Partial match  < 0.40
```

One caveat: `experience_score` is tuned to the challenge job's 5 to 9 year band, so a strong junior candidate is slightly under-rated for every role.

Domain	Roles
AI/ML	Senior AI Engineer, ML Platform Engineer, NLP Researcher
Data	Data Scientist, Data Engineer, Business Analyst
Backend · Frontend · Full stack	Backend Engineer, Frontend Developer, Full Stack Developer
Mobile	Android Developer, iOS Developer, Flutter Developer
DevOps	DevOps Engineer, Cloud Architect, Site Reliability Engineer
Security · Product · Design	Cybersecurity Analyst, Product Manager, UI/UX Designer
Finance · Marketing	Financial Analyst, Digital Marketing Manager

14.7 Resume ZIP ranking

`/screen-bulk` accepts PDF and image files inside the ZIP (ignoring macOS `__MACOSX` entries), extracts each one, and runs the same feature scoring as `/rank`. Semantic scoring is not used in this path and every resume has the same default behavioral signals, so the ranking is driven entirely by the skill composite. The output CSV adds `name` and `source_file` columns so recruiters can find the original files.

14.8 Storage

Uploads and results live under the system temp directory in one folder per job, and job state lives in the `JOBS` dictionary in memory. Nothing is cleaned up explicitly; everything disappears when the process restarts or the container is redeployed. This keeps the design simple but means jobs do not survive a restart and cannot be shared across multiple server instances.

15. Frontend

15.1 Structure

The frontend is a Next.js 16 app whose entire flow lives in one page, `app/page.tsx`. A TypeScript union type `Screen` lists every screen, and a single `useState` decides which component renders. Framer Motion's `AnimatePresence` animates transitions between them. There is no router: the app is a state machine.

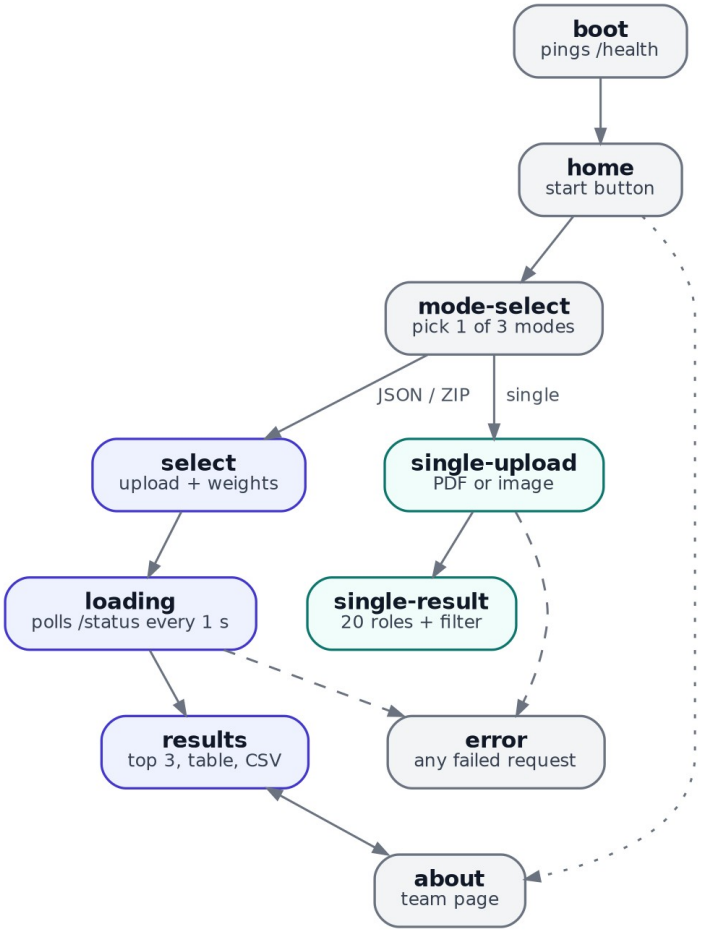


Figure 9. Screen flow. Indigo screens serve the JSON and ZIP modes, teal screens the single-resume mode; dashed arrows are error paths.

15.2 Components

Component	Role
`hand-writing-text.tsx`	Boot animation: an SVG stroke drawn around “Hirer” in the Caveat handwriting font
`background-paths.tsx`	Home screen: 72 animated SVG paths behind the title and the Start button
`radial-orbital-timeline.tsx`	Rotating job selector; the orbit radius is 36% of the container’s shorter side and positions update every 50 ms
`loading-page.tsx`	Progress ring and bar, the last six real pipeline steps, and 21 rotating messages that change every 3.5 s with a typing effect
`display-cards.tsx`	Top-three stacked, skewed cards; rendered #3, #2, #1 so #1 paints on top
`8bit-not-found1.tsx`	Error screen with the 8-bit ogre
`button.tsx`, `badge.tsx`, `card.tsx`	Small UI primitives in the shadcn/ui style

15.3 The polling client

1. Build a `FormData` with the file, the three weights, `top_n` and `use_semantic`.
2. POST it to `/rank` (JSON mode) or `/screen-bulk` (ZIP mode) and read back the `job_id`.
3. Loop: wait one second, GET `/status/{job_id}`, update the percentage and append any new step to the on-screen log.
4. When status is `done`, GET `/preview/{job_id}?limit=100` and switch to the results screen.
5. When status is `error`, or any request fails, switch to the error screen with the message.

The download button simply opens `/download/{job_id}` in a new tab. The single-resume screen makes one synchronous POST to `/screen-single` instead, since that endpoint answers directly.

15.4 Waking the server

`BootScreen` fires `fetch(API + "/health")` when it mounts and ignores the result. Its only purpose is to start the sleeping backend while the user watches the opening animation, hiding most of the cold-start delay.

15.5 The results screen

The results screen shows the top-three cards and the full table. The orbital wheel on the left lets the user pick another of the five challenge-style jobs, which calls `reScoreForJob`. That function multiplies each candidate's score by one of five fixed factors chosen by the candidate's **row position** (cycling every five rows), caps at 0.99 and re-sorts. It is a visual preview only: it uses no candidate attributes, does not re-run the pipeline, and changes the order even for the original job. Section 19 proposes a real replacement.

15.6 Configuration

The backend URL comes from `NEXT_PUBLIC_API_URL`. Next.js inlines `NEXT_PUBLIC_` variables into the JavaScript bundle at **build time**, so changing it on Vercel only takes effect after a redeploy. `next.config.ts` sets `devIndicators: false` to hide the development badge. Fonts (Caveat and Space Mono) load from Google Fonts through a link tag in `layout.tsx`.

16. Deployment and operations

16.1 Current setup

The frontend is deployed on **Vercel**. The backend currently runs on **Render's free tier**: 512 MB of RAM, 0.1 CPU, and a service that sleeps when idle (Render warns that waking can delay a request by 50 seconds or more). Semantic scoring is disabled on the live backend to stay within memory.

16.2 Problems met along the way

Symptom	Cause	Resolution
Loading screen stuck at 0%	The hosting proxy buffered the Server-Sent Events stream	Replaced streaming with background jobs and polling
pandas and sentence-transformers would not install	The platform defaulted to Python 3.14, too new for the pinned packages	Pin `PYTHON_VERSION=3.11.0`
Build failed with “Permission denied” on apt-get	No root access in Render’s native Python runtime, so Tesseract cannot be installed	Planned move to a Docker container on Railway
Local OCR models not an option	512 MB of RAM; RapidOCR alone has been reported using 500 to 700 MB, and EasyOCR and Surya need more	Tesseract (small, CPU-only) inside Docker
requirements.txt not found during build	Build command path did not match the service’s root directory setting	Run the build relative to `hirer-backend/`

16.3 Planned backend: Railway with Docker

A Docker container gives full control over system packages, which is what OCR needs. The Dockerfile below lives in `hirer-backend/`:

```
FROM python:3.11-slim
RUN apt-get update && apt-get install -y --no-install-recommends \
        tesseract-ocr poppler-utils \
    && rm -rf /var/lib/apt/lists/*
WORKDIR /app
RUN pip install --no-cache-dir torch \
        --index-url https://download.pytorch.org/whl/cpu
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
ENV JSONL_ENTRY_LIMIT=1000 PDF_COUNT_LIMIT=50 PDF_SIZE_LIMIT_MB=50
CMD ["sh", "-c", "uvicorn main:app --host 0.0.0.0 --port ${PORT:-8000}"]
```

1. In Railway, create a project from the GitHub repository and set the service’s root directory to `hirer-backend`. Railway detects the Dockerfile.
2. Set `FRONTEND_URL` to the Vercel domain. Railway provides `PORT` automatically.
3. In Vercel, set `NEXT_PUBLIC_API_URL` to the new Railway URL and redeploy the frontend.
4. Check `/health`, then test all three modes, including a scanned PDF and a JPG resume.

16.4 Running locally

```
git clone https://github.com/Sidvortex/Hirer.git
cd Hirer
chmod +x start.sh
./start.sh       # creates a venv, installs everything, starts both servers

# or manually, in two terminals:
cd hirer-backend && python3.11 -m venv venv && source venv/bin/activate
pip install -r requirements.txt && uvicorn main:app --reload --port 8000

cd hirer-frontend && npm install && npm run dev      # http://localhost:3000
```

For OCR on a local machine, install Tesseract and Poppler through the system package manager (for example `sudo pacman -S tesseract tesseract-data-eng poppler` on Arch, or `sudo apt install tesseract-ocr poppler-utils` on Debian and Ubuntu). Offline releases are also published on GitHub: **V2** is the full-stack version and **V1** the CLI-only backend.

17. Testing and quality assurance

17.1 Checks performed for this document

Check	Result
CLI on the 1,000-candidate sample, then `validate_submission.py`	Failed with 3 tie-order errors before the fix; “Submission is valid” after
`/rank` with `JSONL_ENTRY_LIMIT=500` and a 1,000-line file	HTTP 400 with the explanatory message
`/rank` with no limit	Job completed, `/status` reported `done`, `/preview` returned 100 rows
`/screen-single` with a generated NLP-engineer resume PDF	Name, 8 years of experience and skills extracted; best match Senior AI Engineer at 81%
`/screen-bulk` with a two-resume ZIP	Status polling worked, both resumes extracted and ranked with names
Extractor on synthetic resumes with tricky words	Correct years, job dates, institution, tier and CGPA; no false skills from “average”, “excellent” or “submitted”

The backend checks ran in-process with FastAPI’s `TestClient`, which executes background tasks synchronously after each response.

17.2 Recommended manual checklist

Area	What to check
Boot and wake-up	With a sleeping server, the first upload works without a manual refresh
JSON mode	Weights must total 100; a file over the live limit shows the server’s message; CSV download opens; validator passes on the CLI output

Area	What to check
Single resume	Text PDF, scanned PDF, JPG and PNG; a resume with two columns; a resume with no Experience header
Bulk ZIP	Mixed PDFs and images; a ZIP with a `__MACOSX` folder; a ZIP over 50 files on the live server
Errors	Wrong file type, empty file, backend offline: each reaches the error screen with a readable message
Cross-browser	Chrome, Firefox and a phone browser; animations and the orbital wheel stay usable

A natural next step is a small pytest suite covering each feature function with hand-computed expectations (the worked example in Section 11.13 is a ready-made test case), the extractor on a set of fixture resumes, and the API endpoints through `TestClient`.

18. Known limitations and technical debt

Issue	Impact	Suggested fix
CLI and API use different job description texts	The same file can rank differently in the terminal and the web app	Move `JD_TEXT` into one shared module
Scoring modules are duplicated in `src/` and `hirer-backend/`	Fixes must be applied twice and can drift	One installable package imported by both
Recency uses today's date	Results change with the run date; old data looks inactive	Use a fixed reference date, such as the newest `last_active_date` in the file
Semantic scoring off on the live server	35% of the weight is inert; maximum score 0.65	More memory, or precomputed embeddings
Substring keyword matching	False positives such as `cv` in "OpenCV" (47 of 1,000 sample candidates)	Whole-word matching, as already done in the extractor
Embeddings ignore negation	"NOT looking for computer vision" can attract CV profiles	Remove negative phrases from the embedded JD text
Results-screen rerank is a heuristic	Misleading if read as a real ranking	Re-run scoring with a per-job JD and keyword lists
Job wheel on the JSON upload screen is cosmetic	Selecting a job does not change scoring	Pass the chosen job to the backend
Identical behavioral scores in resume modes	25% of the weight is inert for resumes	Drop or re-weight the component for resume inputs
Min-max normalization	Scores are relative and sensitive to outliers and tiny pools	Rank-based or z-score normalization, or report raw component scores

Issue	Impact	Suggested fix
In-memory job store, no cleanup	Jobs lost on restart; temp files accumulate; one instance only	Redis or SQLite for jobs, periodic cleanup
Blocking async job	Status polls wait during long stages	Plain `def` job or a worker queue
Education tier lists	Encode institutional prestige, a possible source of bias	Audit, reduce or remove; report fairness metrics
`submission_metadata.yaml` has placeholder values	Team, contact and repository fields are not filled in	Fill in the real details

19. Roadmap

Short term

- Move the backend to Railway with Docker so scanned and image resumes work live.
- Unify the job description text and merge the duplicated scoring modules.
- Fix the recency reference date and switch keyword matching to whole words.
- Add the pytest suite described in Section 17.

Medium term

- Real per-job ranking: let the user pick or paste a job description, derive keyword lists from it, and re-run scoring instead of the visual rerank.
- Persistent jobs with Redis or SQLite, plus cleanup of old uploads.
- Use the unused signals, especially `willing_to_relocate`, `interview_completion_rate` and `skill_assessment_scores`.
- Precompute candidate embeddings once so semantic scoring fits on small servers.

Long term

- Learning to rank: train a model on recruiter outcomes (shortlisted, interviewed, hired) instead of hand-set weights, and evaluate it with NDCG and MRR, the same metrics the challenge job description mentions.
- Better resume understanding: either LLM-assisted extraction for messy layouts or a small purpose-built OCR model (the ResumeOCR-Lite experiment) that fits in limited memory.
- A fairness audit of education and other proxy signals, with results reported alongside the rankings.

20. Team

Member	Role	Contribution
Ravada Siddharth	ML Engineer / Lead	Ranking pipeline, semantic embeddings, behavioral scoring, full-stack web app

Member	Role	Contribution
Ishan Gupta	Data Engineer	Feature engineering, data loading pipeline, keyword matching, dataset exploration
Ayush Mishra	Frontend / Docs	Frontend interface, presentation slides, project documentation
Vinayak Kapoor	Testing & QA	Testing, validation of submission outputs, candidate schema verification

Repository: github.com/Sidvortex/Hirer

Appendix A. Keyword lists

Must-have (27)

embeddings, sentence-transformers, sentence transformers, vector database, vector search, faiss, pinecone, weaviate, qdrant, milvus, elasticsearch, opensearch, python, ranking, retrieval, ndcg, mrr, information retrieval, nlp, llm, fine-tuning, transformer, bert, learning to rank, hybrid search, bm25, semantic search

Nice-to-have (16)

lora, qlora, peft, xgboost, lightgbm, a/b testing, pytorch, tensorflow, hugging face, langchain, openai, recommendation system, recsys, mlops, kubernetes, docker

Negative signals (7)

computer vision, cv, image classification, object detection, speech recognition, robotics, ocr

Preferred titles (11)

ml engineer, machine learning engineer, ai engineer, nlp engineer, search engineer, data scientist, senior engineer, software engineer, backend engineer, research engineer, applied scientist

Extractor institution tiers (43)

Tier 1 (18): iit, iim, iisc, bits pilani, nit, mit, stanford, harvard, oxford, cambridge, cmu, iit bombay, iit delhi, iit madras, iit kanpur, iit kharagpur, iit roorkee, iit guwahati

Tier 2 (14): vit, manipal, srm, amity, symbiosis, pec, thapar, dtu, nsut, iiit hyderabad, iiit bangalore, iiit delhi, jadavpur, anna university

Tier 3 (11): muit, galgotias, gl bajaj, hbtu, aktu, mtu, sharda, bennett, lovely professional, lpu, maharishi

Note that the extractor's tier list is separate from the challenge data, which already includes a `tier` field for every education entry.

Appendix B. API quick reference

```
# health / wake-up
curl https://<backend>/health

# start a ranking job
curl -F file=@data/candidates_small.jsonl -F skill_weight=0.4 \
     -F semantic_weight=0.35 -F behavioral_weight=0.25 -F top_n=100 \
     https://<backend>/rank

# poll, preview, download
curl https://<backend>/status/<job_id>
curl "https://<backend>/preview/<job_id>?limit=100"
curl -o submission.csv https://<backend>/download/<job_id>

# single resume
curl -F file=@resume.pdf https://<backend>/screen-single

# resume ZIP
curl -F file=@resumes.zip -F skill_weight=0.4 -F semantic_weight=0.35 \
     -F behavioral_weight=0.25 -F top_n=50 https://<backend>/screen-bulk
```

Interactive documentation for every endpoint is generated automatically at `/docs` (Swagger UI).

Appendix C. Glossary

Term	Meaning
API	The set of web addresses (endpoints) the frontend calls to ask the backend to do work
Backend / frontend	The server that does the processing / the website the user sees
JSONL	A text file with one JSON record per line; the challenge's candidate format
CSV	A plain-text table that opens in Excel or Google Sheets
Embedding	A list of numbers representing the meaning of a text; similar meanings give nearby lists
Cosine similarity	A measure of how closely two embeddings point in the same direction, from −1 to 1
Semantic	About meaning rather than exact wording
Min-max normalization	Rescaling values so the lowest becomes 0 and the highest becomes 1
OCR	Optical character recognition: reading text out of an image
Tesseract	A free, widely used OCR engine
Polling	Asking the server "are you done yet?" at a regular interval
SSE	Server-Sent Events: the server pushes updates over one long-lived connection
Job ID	A random identifier for one processing run

Term	Meaning
CORS	Browser security rules about which websites may call an API
Cold start	The delay while a sleeping free-tier server boots up
Docker	A way to package an app with everything it needs, including system software
Environment variable	A setting passed to a program from outside its code
Tie-break	The rule for ordering items with equal scores
NDCG, MRR	Standard metrics for judging the quality of a ranked list